arXiv:2512.22560v1 [cs.DC] 27 Dec 2025

ROLL

ROLLART: Scaling Agentic RL Training via Disaggregated Infrastructure

Wei Gao[†*], Yuheng Zhao[†*], Tianyuan Wu[†*], Shaopan Xiong[‡*], Weixun Wang[‡*], Dakai An[†], Lunxi Cao[†], Dilxat Muhtar[‡], Zichen Liu[‡], Haizhou Zhao[‡], Ju Huang[‡], Siran Yang[‡], Yongbin Li[¶], Wenbo Su[‡], Jiamang Wang[‡], Lin Qu[‡], Bo Zheng[‡], Wei Wang[†]

[†]HKUST [‡]Alibaba Group [¶]Tongyi Lab, Alibaba

Abstract

Agentic Reinforcement Learning (RL) enables Large Language Models (LLMs) to perform autonomous decision-making and long-term planning. Unlike standard LLM post-training, agentic RL workloads are highly *heterogeneous*, combining compute-intensive prefill phases, bandwidth-bound decoding, and stateful, CPU-heavy environment simulations. We argue that efficient agentic RL training requires *disaggregated infrastructure* to leverage specialized, best-fit hardware. However, naive disaggregation introduces substantial synchronization overhead and resource underutilization due to the complex dependencies between stages.

We present ROLLART, a distributed system designed to maximize throughput for multi-task agentic RL on disaggregated infrastructure. ROLLART is built on three core principles: (1) *hardware-affinity workload mapping*, which routes compute-bound and bandwidth-bound tasks to best-fit GPU devices, (2) *fine-grained asynchrony*, which manages execution at the trajectory level to mitigate resource bubbles, and (3) *statefulness-aware computation*, which offloads stateless components (e.g., reward models) to serverless infrastructure for elastic scaling. Our results demonstrate that ROLLART effectively improves training throughput and achieves 1.35-2.05× end-to-end training time reduction compared to monolithic and synchronous baselines. We also evaluate ROLLART by training a hundreds-of-billions-parameter MoE model for Qoder product on an Alibaba cluster with more than 3,000 GPUs, further demonstrating ROLLART's scalability and robustness. The code is available at https://github.com/alibaba/ROLL.

1 Introduction

Reinforcement Learning (RL) has become a cornerstone for advancing Large Language Models (LLMs) from passive logical reasoning toward autonomous decision-making and long-horizon planning [1, 9, 44]. This paradigm, known as *agentic RL*, requires LLMs to operate effectively within complex, dynamic environments, ranging from tool use [18, 61] to web navigation [25, 27, 54] and general computer control [32, 34, 37]. Unlike traditional LLM post-training, agentic RL emphasizes learning through extensive practice: the agent actively interacts with external environments to generate *long, multi-turn trajectories*, thereby solving complex reasoning tasks through trial and error.

The agentic RL training pipeline operates as an iterative cycle comprising three stages: *rollout*, *reward*, and *training*. During rollout, the agent LLM engages in a *multi-turn feedback loop* with an environment, generating action tokens and receiving observations until a termination condition is met, which constitutes a *trajectory*. These trajectories are then evaluated in the reward stage, often by separate models or code sandboxes, to assign scalar reward signals. Finally, the training stage consumes these labeled trajectories to update the agent LLM's weights. These updated parameters are then synchronized back to the rollout workers for the subsequent iteration, creating a loop of data generation and policy update.

As research institutes investigate *Scaling Laws for Agentic RL* [10, 58], a fundamental infrastructure challenge emerges: the resource requirements across the pipeline are highly *heterogeneous* and often *conflicting*. The **rollout stage** alone presents a complex composite workload. LLM generation alternates between TFLOPS-hungry prefill phases—best served by compute-optimized hardware (e.g., NVIDIA H800)—and bandwidth-bound decoding phases that benefit from high memory bandwidth (e.g., H20). The ratio of these phases is dictated by the task structure: long-horizon tasks like `FrozenLake` [11] or `SWE-bench` [26] are prefill-heavy, whereas short-turn tasks like `GEM-math`/`GEM-game` [5] are decoding-dominant. Concurrently, the agent LLM must interact with thousands of parallel environments. These environments are often *stateful*, *CPU-bound simulations* that introduce severe resource contention if colocated on GPU nodes. In contrast, the **reward stage** typically consists of *stateless, parallelizable* workers running diverse evaluations from sim-

[*]Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, and Weixun Wang contributed equally to this work.

ple CPU scripts [15, 21] to complex GPU-based model judgments [53, 70]. Finally, the **training stage** demands high-end GPUs with fast interconnects to sustain the high TFLOPS required for large-scale parameter optimization.

We contend that a single monolithic cluster cannot efficiently satisfy the diverse, conflicting resource requirements of multi-task agentic RL training. While a natural solution is to exploit *disaggregated architectures* that route stages to *best-fit* hardware, current systems fail to fully realize this potential. Industry-standard systems such as veRL [50, 51], slime [72], and rLLM [57] remain bound to monolithic GPU clusters. Newer frameworks like AWorld [65] and DeepSWE [35] introduce *partial disaggregation* by offloading environments to Kubernetes [4] clusters. However, they still colocate the resource-heavy rollout and training stages, leaving the resource mismatch problem unresolved. Even recent system efforts that decouple training from rollout, such as StreamRL [69], AsyncFlow [17], SeamlessFlow [59], and Laminar [49], adopt a *coarse-grained* approach. These systems largely overlook the inherent heterogeneity of agentic tasks and environments, limiting their ability to fully exploit specialized hardware for expedited rollout.

While disaggregation enables hardware specialization, it introduces severe orchestration challenges due to *heterogeneous communication patterns* and *synchronization overheads*. First, the data transfer requirements are highly *unbalanced*. Weight synchronization between training and rollout clusters is bandwidth-intensive, involving bulk data transfer of hundreds of gigabytes and incurring latency on the order of tens of seconds. Conversely, trajectory transfers between the agent LLM and environments are smaller (kilobytes to gigabytes) but suffer from extreme variance: environment instability can introduce *long-tail latencies* reaching hundreds of seconds (Figure 5a), severely blocking dependent stages. Second, in synchronous training, these communication delays create "resource bubbles" (Figure 2-Left), where expensive training GPUs idle while waiting for straggling environments or weight updates. While one could attempt stitching together current disaggregated RL frameworks [17, 59, 69] with Kubernetes-orchestrated agentic environments, such ad-hoc integration can only serve for a single job with dedicated, high-bandwidth networks and does not scale to production-grade, multi-tenant clusters where cross-cluster links often exhibit dynamic, heterogeneous bandwidth.

To address the aforementioned challenges, we present ROLLART, a distributed system designed to maximize throughput for multi-task agentic RL on disaggregated infrastructure. ROLLART evolves from our prior technical reports on ROLL [60] and ROLL-Flash [33]. Unlike systems that treat the cluster as a uniform pool, ROLLART orchestrates the RL pipeline according to three core design principles. **(1) Hardware-Affinity Workload Mapping:** ROLLART allows users to bind an entire stage to a dedicated hardware resource pool (e.g., constraining training to compute-optimized GPUs). It also supports *fine-grained affinities* within a stage. This allows, for example, routing prefill-heavy, compute-bound tasks (e.g., `FrozenLake` and `SWE-bench`) to H800 GPUs while scheduling decoding-dominant, bandwidth-bound tasks (e.g., `GEM-math`) on H20 GPUs. The hardware affinity can fully exploit available resources and maximize the system throughput. **(2) Fine-grained Asynchrony:** Within rollout, this principle advocates operating at the *trajectory level* rather than the batch level. By interleaving environment interaction, LLM generation, and reward computation, the system effectively masks inter-cluster communication latency and prevents slower environment stragglers from stalling the entire pipeline. For asynchronous training, it requires fine-grained control of the asynchrony between rollout and training to balance gradient stability and system throughput. **(3) Statefulness-aware Computation:** To optimize resource efficiency, ROLLART explicitly categorizes components by their state requirements. It offloads inherently stateless components, such as reward models, to serverless infrastructure (e.g., Function Compute [3]), granting the system zero-overhead autoscaling, multi-tenancy, and fault tolerance without dedicated, hot-standby GPUs.

We realize these principles in ROLLART, a distributed system that combines a declarative programming model with a heterogeneity-aware runtime. Through Python-based decorators, users explicitly define *hardware affinities* for specific sub-tasks and register stateless components to external serverless interfaces. Under the hood, ROLLART's resource manager allocates heterogeneous resources to specialized *workers*, while the distributed runtime orchestrates an asynchronous, trajectory-level workflow. To maximize hardware efficiency, the runtime integrates optimized execution engines, using inference engines [2, 28, 45] for high-throughput generation and training engine [52] for distributed training. The runtime also exposes multiple transfer channels that exploit the underlying heterogeneous network fabric (NVLink, InfiniBand, Ethernet), optimizing both the stability-critical trajectory transfer and bandwidth-intensive weight synchronization.

We implemented ROLLART from scratch in approximately 60k lines of Python code. We evaluate ROLLART by training multiple Qwen models [8] on disaggregated clusters, demonstrating that it achieves up to a 2.05× speedup over monolithic and synchronous baselines. To validate scalability and stability in a production setting, we deploy ROLLART to train a large Mixture-of-Experts (MoE) LLM for Qoder [42] product. This deployment, running continuously for one week on a cluster of over 3,000 GPUs, confirms ROLLART's ability to sustain high throughput and robust fault tolerance at scale.

2 Background

2.1 Agentic RL Training

The Training Pipeline. The training pipeline for multi-task agentic RL is an iterative loop comprising three distinct

Table 1: Taxonomy of Adopted Agentic Environments.

Environment	Task Domain	Modality	#Turns
SWE-bench [26]	SWE	Text	30–50
WebShop [63]	Web	Text	5–30
FrozenLake [11]	Game	Text, Visual	20–100
GEM-math [5]	Math+Tool Use	Text	< 5
GEM-game [5]	Game	Text	1

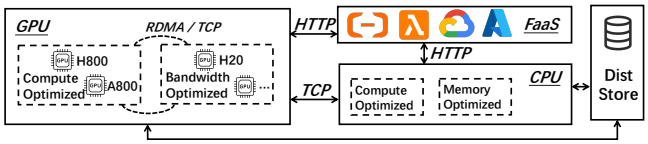


Figure 1: Disaggregated infrastructure for agentic RL training.

computational stages. The first stage is a **rollout** for experience collection. In this stage, the agent LLM (actor) interacts with parallel *environments* to generate training data. Unlike standard LLM inference, this process is *multi-turn* and *stateful* [10, 58]. In each turn, the agent observes a state, generates an action token sequence, and submits it to the environment. The environment executes the action (e.g., running code, clicking a link) and returns feedback. This cycle repeats until a termination condition is met, producing a sequence of state-action pairs known as a *trajectory*. Once a trajectory is complete, the system proceeds to a **reward stage**, where a *reward worker* evaluates the quality of the agent's actions. This evaluation yields a scalar reward signal, computed via methods ranging from lightweight rule-based checks [21] to computationally intensive model-based judgments (e.g., LLM-as-a-Judge [53, 70]). Finally, the collected trajectories and rewards are consumed by the **training stage** to update the agent LLM's weights using RL algorithms (e.g., PPO [43], GRPO [48]). For training performance [12], RL researchers typically adopt *synchronous* RL training which requires strict synchronization of the model weights between the rollout and training stage in each step.

Environment Heterogeneity A key challenge in agentic RL is the extreme diversity of the environments, which dictates the system's compute profile. As summarized in Table 1, agentic tasks vary significantly in modality and interaction frequency. Complex reasoning tasks like `SWE-bench` [26] (software engineering), `FrozenLake` [11] (visual games), and `WebShop` [63] (eCommerce) require long interaction horizons (up to 30–100 turns). Frequent interactions force the agent LLM to repeatedly process the growing context history, making the workload prefill-heavy and computationally intensive. Conversely, tasks like `GEM-Math` and `GEM-Game` [5] may involve fewer turns (up to five) but require generating longer chains of thought per action. These workloads are decoding-heavy, shifting the bottleneck from compute to memory bandwidth. This variance means that the infrastructure must adapt to the specific characteristics of the current task's rollout.

Table 2: NVIDIA GPU specifications.

Hardware Specification	H20	H800
TFLOPS	148	989.5
HBM capacity	96GB	80GB
HBM bandwidth	4TB/s	3.35TB/s
NVLink bandwidth	900GB/s	400GB/s
Normalized Cost [71]	1.00	2.85

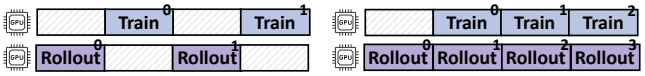


Figure 2: Synchronous vs. asynchronous training.

2.2 Disaggregated Cluster Infrastructure

The Case for Disaggregation. The extreme heterogeneity of agentic RL workloads, ranging from compute-intensive training to stateful environment simulation (§2.1), renders monolithic architectures inefficient. Consequently, agentic RL training must transition to a *disaggregated infrastructure* that decouples these demand-conflicting computation stages into specialized resource pools. As illustrated in Figure 1, in a typical disaggregated infrastructure, *training clusters* utilize high-end, compute-optimized GPUs (e.g., NVIDIA H800) for massive throughput; *inference clusters* leverage bandwidth-optimized hardware (e.g., H20) to serve memory-bound decoding; *CPU clusters* provide elastic capacity for diverse, containerized runtime environments orchestrated by Kubernetes [4]; and *serverless infrastructure* handles bursty, stateless workloads like reward evaluation. These pools are interconnected via standard network fabrics, relying on distributed storage for persistent logging and fault tolerance.

Sync. vs. Async. Training. While disaggregation resolves resource mismatches, it introduces non-trivial orchestration challenges to the training paradigm that dictates the trade-off between system throughput and algorithmic consistency.

1) Synchronous Training: This paradigm enforces strict consistency by blocking the rollout stage until the latest model weights are received from the training cluster. In a disaggregated setting, however, this introduces substantial "dependency bubbles" (Figure 2-Left), where expensive GPUs sit *idle* during the high-latency weight synchronization and straggler-bound environment steps (§3).

2) Asynchronous Training: To mitigate these bubbles, systems can adopt *asynchronous* paradigms (e.g., one-off RL training [36] as shown in Figure 2-Right). Here, rollout and training execute *in parallel*: the training stage consumes trajectories generated by slightly older policies (e.g., one iteration behind in one-off training), while the rollout stage continuously produces new data. This approach effectively masks the synchronization latency and straggler effects inherent to disaggregation, trading a degree of policy freshness (staleness) for maximized hardware utilization and system throughput.

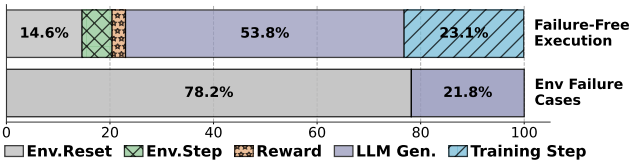
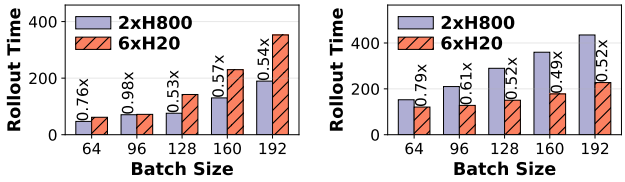


Figure 3: Breakdown of a training step: successful runs (top, avg=365.7s) versus execution with environment failures (bottom, avg=513.3s).



(a) `FrozenLake` [Prefill-Heavy]. (b) `GEM-Math` [Decode-Heavy].

Figure 4: End-to-end rollout time (seconds) of different tasks on H20 and H800 GPUs across varying batch sizes.

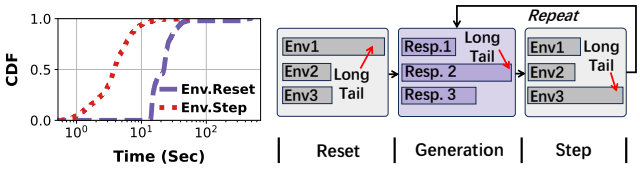
3 Characterization and System Requirements

To motivate the design of ROLLART, we conduct a comprehensive workload characterization of multi-task agentic RL training. Based on these empirical observations, we derive a set of critical system requirements (summarized in the boxes below) that ROLLART must satisfy.

3.1 Stage Computation

Training Step Latency Breakdown. We first profile the end-to-end latency of a standard training iteration to identify the dominant cost components. We train Qwen3-8B/32K on 32 H800 GPUs using the `SWE-bench` environment (batch size 128), where the agent LLM interacts with a containerized sandbox for software engineering tasks. The interaction involves two core operations: `env.reset` for environment initialization via Docker image pulling and container launching, and `env.step` for agent's action execution.

Figure 3 breaks down the latency of five *successful* iterations against five iterations containing *environment failures*. In successful runs, the average iteration time is 366 seconds, with LLM generation dominating (54%), followed by training (23%) and environment initialization (15%). However, in iterations where environment timeouts due to failures, the average time spikes to 513 seconds. Crucially, in these failure scenarios, `env.reset` alone consumes 78% of the rollout time, shifting the bottleneck entirely from GPU computation to environment overhead. Our production data indicates these failures are not rare corner cases, occurring approximately once every ten iterations. While prior works [14, 20] report that LLM generation dominates RL runtime (>80%), our empirical study reveals that in agentic RL, efficient management of the environment lifecycle is the paramount challenge.



(a) Env Time Distribution. (b) Batched Env Interaction.

Figure 5: The analysis of environment interaction: (a) Cumulative distribution function of time (log-scaled) taken for environment initialization (`env.reset`) and environment step (`env.step`). (b) Illustration of how long-tail environments affect multi-turn rollouts under batched env interaction.

Divergent Hardware Affinities in Generation. Modern GPUs expose different trade-offs between compute capability, memory capacity, and cost (Table 2). While recent RL systems [17, 59, 69] advocate physically decoupling generation from training—assigning rollout to cost-effective, bandwidth-optimized GPUs (e.g., NVIDIA H20) and training to compute-optimized devices (e.g., H800)—our characterization reveals that *this static assignment is insufficient for agentic workloads*. LLM generation comprises two distinct phases with opposing resource demands: the compute-bound prefill phase and the memory-bandwidth-bound decoding phase. In environments with a few interaction turns and reasoning thinking pattern, most of the runtime is spent in decoding (decoding-heavy); conversely, in environments with many turns, the prefill phase dominates and demands high compute throughput (prefill-heavy). In our production clusters, agentic RL tasks exhibit a clear *bimodal distribution*, featuring either a small number of interaction turns (< 5) or a large number (> 10).

To quantify this divergence, we run a prefill-heavy task (`FrozenLake`) and a decoding-heavy task (`GEM-Math`) using Qwen3-8B/32K for ten iterations with prefix caching enabled. For a cost-equivalent comparison, we execute the workloads on two distinct hardware configurations: one with 6 H20 GPUs and the other with 2 H800 GPUs. As shown in Figure 4a, the compute-dense H800 outperforms the H20 on `FrozenLake`, reducing end-to-end rollout time to as low as 0.53×. Conversely, for `GEM-Math` (Figure 4b), the H20's superior memory bandwidth expedites the decoding phase, the H20's higher memory bandwidth accelerates decoding, reducing rollout time to 0.49×–0.79× of the H800. These results invalidate the dogmatic assumption that generation is uniformly bandwidth-bound. Instead, maximizing throughput requires dynamically mapping tasks to their best-fit hardware.

> **R1:** The optimal resource allocation of LLM generation should align workload characteristics (e.g., prefill- vs. decoding-heavy, batch size) with hardware capabilities.

Heavy-Tailed Environment Execution. Our production experience running thousands of concurrent environments warrant that resource isolation without interference is essential.

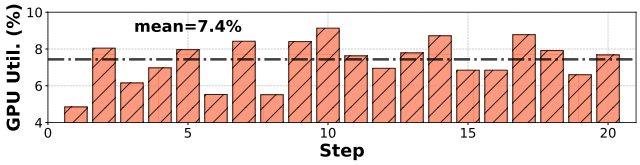


Figure 6: Inefficient resource usage when dedicating local GPUs for reward computation.

Resource sharing can lead to severe issues, such as concurrent disk I/O exhausting shared quotas and causing cascading failures. Consequently, we employ dedicated Kubernetes clusters to manage and isolate each containerized environment.

Substantial prior systems [14, 20, 69] have analyzed the long-tail behavior of LLM generation. Figure 5a illustrates a similar long-tail distribution in the latency of `env.reset` and `env.step` operations, with the effect particularly pronounced for `env.reset`. In our internal production deployments, the long-tail delay of `env.reset` can reach hundreds of seconds, mainly due to two factors: (1) network contention, where hundreds of environment workers simultaneously fetching docker images can saturate network links and overwhelm the image registry service, and (2) compute and I/O contention on host nodes, where launching new containers consumes substantial CPU and disk resources.

The long-tail environments would become stragglers and delay the end-to-end rollout latency. Specifically, the agent LLM interacts with multiple environments *simultaneously*. Since underlying LLM engines execute requests in batches, it is natural to batch environment interactions with the agent LLM as well. Figure 5b illustrates this batched execution pattern, in which the long-tail environment behavior can significantly increase end-to-end rollout latency. Fast environments are forced to wait for the slowest ones before the next generation step can proceed. Our profiling (Figure 3) indicates that batched environment interaction increases rollout time by up to 21.3% compared to ideal execution, an overhead that compounds as environment failure rates increase.

R2: Environment execution, including `env.reset` and `env.step`, is prone to extreme long-tail latency. To prevent stragglers from stalling the pipeline, the system must abandon batched environment interaction in favor of fine-grained, asynchronous environment management.

Stateless Reward Computation. The reward stage follows the rollout stage, and most reward computations can be implemented as *stateless functions*. For lightweight code-based and rule-based rewards, the resource demand is relatively small, making it natural to offload these workers on a serverless platform for scalable and low-latency computation. The LLM-based reward computation demands intensive GPUs, contending with rollout workers for GPU resources. A common practice is to reserve dedicated GPUs for reward LLMs. As an example, in our Qwen3-8B/32K `SWE-bench` experiment with a batch size of 128, we allocate 4 H800 GPUs to a dedicated 7B reward LLM and 28 H800 GPUs to rollouts. However, the reward GPUs achieve only 7.4% average utilization across steps (Figure 6). Since the reward LLM's parameters remain fixed during training, it can be treated as a stateless function, which makes serverless deployment [13, 66, 67] a promising approach to improving resource utilization.

Table 3: Transmission overhead from the training cluster to the inference cluster over TCP and RDMA.

Model	Size (GB)	TCP (s)	RDMA (s)	Speedup
Qwen3-8B	15.26	6.911	5.466	1.264×
Qwen3-14B	27.51	14.437	5.817	2.482×
Qwen3-32B	61.02	29.649	9.442	3.140×

R3: Reward workers are stateless and typically exhibit low utilization, making them well-suited for serverless deployment to improve overall resource efficiency.

3.2 Inter-Stage Communication

Beyond computation, inter-stage communication is a critical role in agentic RL training, comprising two distinct types: stability-critical, small-packet *trajectory transfer* and bandwidth-intensive, large-volume *weight update*.

Stability-Critical Trajectory Transfer. The transmission overhead of trajectory data is small compared with the overall training time. In particular, frequent environment interactions are more likely to become the performance bottleneck due to network latency. In practice, environment interaction should prioritize *network stability* over *network bandwidth*. Asynchronous execution between environment interaction and LLM generation is an effective way to prevent from becoming this bottleneck.

Bandwidth-Intensive Weight Update. During training, the agent LLM periodically updates its weights, which must then be synchronized with the rollout stage. This weight synchronization is the dominant source of inter-stage communication overhead. We measure the end-to-end transmission cost of synchronizing model parameters between the training and inference clusters using Mooncake [41] over TCP (200 Gbps Ethernet) and RDMA (400 Gbps InfiniBand), and report the results in Table 3. RDMA provides higher bandwidth and lower communication overhead than TCP. In *synchronous* RL training, the rollout stage can only proceed after the latest agent LLM weights have been synchronized. As a result, the substantial cost of weight transmission over low-bandwidth links can increase end-to-end training time and diminishes the speedup of disaggregated training (Figure 2-Left).

In asynchronous training (Figure 2-Right), the training and rollout stages execute in parallel on separate GPUs. Although this introduces data staleness, many prior works [12, 20, 33] empirically observe that asynchronous training can preserve model quality even under a high data staleness. Given the

dominant rollout overhead, asynchronous training can effectively hide both training and weight synchronization costs with rollout, thereby reducing end-to-end training latency.

R4: Asynchronous RL post-training is desired to improve training efficiency, particularly in disaggregated setups.

4 Design Principles

We present our design principles to meet above requirements for multi-task agentic training on disaggregated infrastructure.

4.1 P1: Hardware-Affinity Workload Mapping

Our first principle is to map workloads to resources based on their *hardware affinity* to meet requirement **R1** (highlighted in the box of §3.1). Users are allowed to define a collection of hardware types at the granularity of both stages and individual trajectories. At the stage level, the training stage can be executed on GPUs, while environment can run on CPUs. At the trajectory level, this principle enables even finer optimization. For example, within the rollout stage, trajectories whose generation is dominated by prefill and does not hit memory limits can be routed to H800 GPUs, whereas trajectories dominated by decoding can be scheduled on H20 GPUs (§3.1). Similarly, within the reward stage, the reward LLM can run on GPUs, while code-sandbox execution can run on CPUs. The fine-grained mapping achieves maximized efficiency.

4.2 P2: Fine-Grained Asynchronous Execution

To comply with **R2** and **R4**, we introduce our second principle: the diverse communication patterns of disaggregated agentic training can be addressed with fine-grained asynchrony. Particularly, it should realize the following designs.

Trajectory-level Rollout Scheduling. Conventional approaches [50] perform LLM generation, environment interaction, and reward computation in a batched manner, which leads to substantial resource wastage and long rollout latency (§3.2). In contrast, trajectory-level scheduling creates a continuous pipeline in which LLM generation for one trajectory overlaps with environment interaction for another and reward computation for a third. This pipelined execution reduces resource idleness and hides environment communication delays, improving resilience to environment instability.

Managed Rollout-Train Synchronization. Asynchronous RL training decouples the rollout and training stages, allowing them to proceed in parallel. The rollout stage continuously produces trajectories, while the training stage consumes these trajectories and periodically synchronizes updated model weights with the inference workers. Our managed synchronization mechanism exposes explicit control over the rate at which completed trajectories are consumed. This control allows practitioners to flexibly configure the asynchronous bound (defined in §5.3) to balance the training stability, system throughput, and model weight synchronization overhead.

4.3 P3: Statefulness-Aware Computation

The third principle satisfies **R3**: we perform statefulness-aware computation. A stateless system component's output depends solely on its input, rendering each execution independent and thus ideal for optimization on a serverless platform.

Rollout: LLM generation. Production systems typically deploy LLM generation using a serverful architecture to achieve low latency and high throughput. Recently, Tinker [29] explores elastic serving for RL rollouts, but is designed around LoRA [23], whereas full-parameter training is more urgent for production-grade LLMs.

Rollout: Environment. This stage is inherently stateful, as the environment (for example, a coding workspace, web browser, or game) is updated by each action. All actions in a trajectory must be routed to the same persistent environment instance, requiring session affinity.

Stateless Reward. A reward worker takes a trajectory as input and produces a scalar value without retaining any memory of past evaluations. This property makes it well suited to a serverless computation model, enabling shared, multi-tenant *Reward-as-a-Service* that can scale elastically to and from zero, thereby maximizing resource utilization.

Training. This stage is inherently stateful, necessitating a dedicated GPU cluster.

5 Programming and Computation Model

We introduce ROLLART, which realizes above design principles via following programming and computation model.

5.1 Declarative Programming Model

In the runtime, a worker is the minimum unit to own resources. Listing 1 shows the programming model defined at the worker function level, giving the runtime fine-grained visibility into resource usage. The details are as follows.

Single Controller. This programming model is widely adopted in industrial RL frameworks [50, 72] as it streamlines the construction of agentic RL pipelines. ROLLART adopts this model and realizes it via the `register` decorator. When the execution mode is set to `execute_all` (Lines 7-8 in Listing 1), the runtime broadcasts inputs to all trainer workers and invokes `compute_gradients` on each worker. ROLLART manages distributed computation and communication across workers, while users implement the computation logic inside each execution function and compose the agentic training pipeline by invoking execution functions of different types of workers.

Hardware-Affinity Mapping (Principle 1). ROLLART supports provisioning heterogeneous resource groups through a

```python
import rollart.distributed as rdist
from rdist.worker import AcrtorTrainCls, ActorGenCls,
     RewardCls
from rdist import ResourceManager as RM

# 1. Single Controller Example
class MyActorTrain(ActorTrainCls):
    @rdist.register(mode="execute_all")
    def compute_gradients(self, input_tensor):
        ...

# 2. Define actor_gen on heterogeneous GPUs
# 2.1 heterogeneous GPU allocation.
gen_rm=RM( {"H800": list(range(0, 8))},
           {"H20", list(range(8, 32))})
# 2.2 hardware affinity mapping
class HeteroActorGen(ActorGenCls):
    @rdist.hw_mapping(
    hw_affinity={"FrozenLake": "H800","default": "H20"}
    )
    def generate(self, input_ids:List[int],
                 tag_name:str="default"):
        return self.model.process(prompt)

# 3. Define a serverless reward computation func
class ServerlessRewardWorker(RewardCls):
    @rdist.register_serverless(
        attribute='reward_proxy',
        serverless_url='fc://xxx.xxx')
    def compute_rewards(self, traj: list):
        prompt = f"Evaluate the trajectory:{traj}"
        return ray.get(self.reward_proxy(prompt))
```

Listing 1: The declarative programming model of ROLLART.

dictionary-based resource specification (Lines 13–14). The detailed illustration of the resource manager can be found in §6. We use the `generate` function in the `HeteroActorGen` class (Lines 16-22) to illustrate how to declare fine-grained, affinity-aware hardware mappings for a worker method via the `hw_mapping` decorator (Lines 17–19). Specifically, LLM generation workloads tagged as `FrozenLake` are routed with high priority to compute-optimized GPUs, while other workloads are routed with high priority to bandwidth-optimized GPUs. The `tag_name` argument (Line 21) serves as a dynamic routing hint, allowing each LLM generation request to be automatically dispatched to workers bound to the appropriate hardware. This routing mechanism ensures that each trajectory generation executes with effective resource affinity.

Serverless Registration (Principle 3). ROLLART exposes `register_serverless` (Lines 26-28) to declare the statefulness of an execution method. This decorator allows the runtime to invoke `compute_rewards` (Line 29) as a pure function on a serverless worker pool via the specified `serverless_url`. The serverless platform automatically manages resource provisioning, scaling, and request routing to achieve high resource efficiency.

5.2 Key Abstraction: `Cluster`

The above programming model simplifies the development of agentic RL training. We next discuss how it is implemented via our `Cluster` abstraction (in Listing 2). Specifically, this abstraction is implemented to serve as a controller, managing the execution of distributed workers across RL stages.

```python
class Cluster:
    def __init__(self, res_manager, worker_cls):
        self._create_worker(worker_cls, res_manager)
        self._bind_worker_method()

    def execute_all(self, method_name, *args, **kwargs):
        result = []
        for worker in self.workers:
            rcall = getattr(worker, method_name)
            result.append(rcall(*args, **kwargs))
        return ray.get(result)

    def hw_mapping(self, hw_affinity, tag_name, *args):
        hw_type = hw_affinity.get(tag_name)
        new_workers = []
        for worker in self.workers:
            if worker.resource_type == hw_type:
                new_workers.append(worker)
        # route requests to new_workers next

    def register_serverless(self, attr, url, *args):
        # define a call_fc to call serverless url
        for worker in self.workers:
            setattr(worker, attr, call_fc)
        # perform execute_all logic next
```

Listing 2: The simplfied implementation and of `Cluster` and its running example in ROLLART.

Distributed Workers. The `Worker` serves as the fundamental execution unit in ROLLART. As a base class, it can be specialized into various computational units for different roles (e.g., training, generation) required across the stages. The `Cluster` instantiates a set of workers according to the specified `worker_cls`, and uses the resource manager to allocate resources to each worker and label their resource types for hardware affinity mapping (Line 3 in Listing 2). The resource manager owns and manages a collection of resources.

Invocation Proxy. The `_bind_worker_method` function binds each method of the specified `worker_cls` to the `Cluster` instance, allowing the `Cluster` to act as a proxy for the corresponding collection of workers (Line 4). For example, if `worker_cls` defines a method `compute_gradients`, users can directly invoke `Cluster.compute_gradients`.

We now describe the computation model underlying our programming model. First, to realize the single controller mechanism, when users invoke a method annotated with the `register` decorator, ROLLART enters the `execute_all` method, which calls the target method on all workers and aggregates their results (Liness 6-11). Second, for methods annotated with `hw_mapping`, ROLLART inspects the `tag_name` argument, filters for workers bound to the preferred resource type, and executes the method on those selected workers (Lines 13–19). Third, for methods annotated with `register_serverless`, the reward proxy (`attr`) is replaced with the registered serverless url, so reward computation is performed by a serverless function (Lines 21–25). We discuss prefill–decoding disaggregation and asynchronous cross-cluster weight update next.

Prefill–Decoding Disaggregation. Many systems [40,68] disaggregate prefill and decoding across heterogeneous GPUs. During rollout, the `Cluster` allows distributed workers to

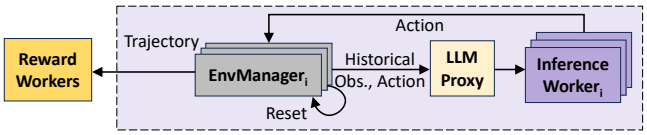


Figure 7: Trajectory-Level Rollout Overview.

leverage these engines to realize such disaggregation. However, real deployments currently require manual configuration of prefill and decoding instances, which easily leads to load imbalance. We hence leave it as future work.

Asynchronous Cross-Cluster Weight Update. Hardware-affinity-driven disaggregation requires cross-cluster communication between the rollout (e.g., H20) and training (e.g., H800) clusters, which may be physically separated and connected only via low-bandwidth Ethernet links. This slow interconnect can become a bottleneck for weight update in frameworks designed for a single, uniform high-bandwidth network [50]. ROLLART addresses this by implementing an asynchronous weight update engine using Mooncake [41]. In our design, training workers (on the H800 cluster) asynchronously publish weights to the Mooncake store without waiting for the concurrent rollout stage to finish, while inference workers (on the H20 cluster) can fetch them on-demand. Then, we follow the existing approach [50] to perform model update within clusters. By effectively hiding cross-cluster communication overhead behind ongoing trajectories, the asynchronous cross-cluster weight update can reduce the end-to-end training overhead.

5.3 Asynchronous Workflows

We describe the asynchronous workflows in ROLLART that realize fine-grained asynchrony (**Principle 2**). Within rollout, LLM generation is overlapped with environment execution. Across stages, rollout is overlapped with reward and training.

`LLMProxy`: Trajectory-Level LLM Generation. `LLMProxy` is a gateway that decouples LLM serving clients from the underlying serving instances. It intelligently orchestrates requests across a fleet of internal LLM inference workers. Each worker is built around a command-driven event loop that manages an inference engine (e.g., vLLM [28], SGLang [45]).

This loop runs continuously in a non-blocking fashion with two components: (1) *Step-wise command processing.* The loop continuously polls for commands dispatched from `LLMProxy`, `ADD` to enqueue new requests and `ABORT` to cancel existing ones. When no commands are pending, it advances the inference engine by executing a single decode or prefill step for a batch of requests, keeping GPU utilization high. This design ensures that adding or aborting an ongoing trajectory does not stall the entire LLM generation process. (2) *Post-processing.* When the LLM engine finishes a request after a certain prefill/decoding step, the loop immediately invokes a pre-registered callback. This callback post-processes the output and returns the result to the original client (e.g., an `EnvManager`). This allows each trajectory to perform environment interaction as soon as its LLM generation completes, without waiting for stragglers. Together, both functions enable trajectory-level LLM generation.

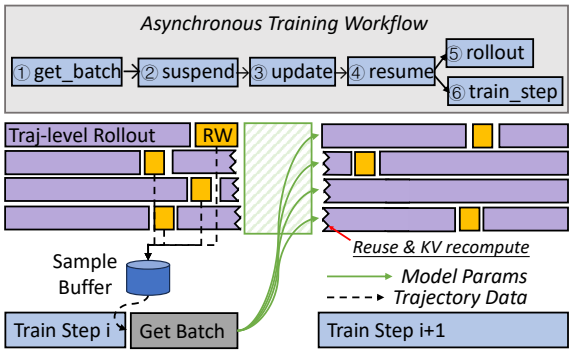


Figure 8: Asynchronous Training Workflow.

`EnvManager`: Trajectory-level Environment Interaction. Each `EnvManager` is a lightweight controller that manages the lifecycle of a single environment to collect trajectories, as shown in Figure 7. It begins with environment initialization via `reset`, after which it enters an independent event loop that orchestrates the interaction between an environment instance and the shared `LLMProxy`. During this loop, the `EnvManager` maintains a list of (observation, action) pairs to construct a trajectory. Specifically, it feeds `LLMProxy` with the historical (observation, action) sequence as input to obtain the next action, applies this action to the environment via `step`, and records the resulting observation.

In practice, ROLLART launches multiple `EnvManager` instances simultaneously, and each `EnvManager` yields a single trajectory rather than batch-executing environment interactions (as shown in Figure 5b). As a result, long-tail environment workers do not delay the execution of other workers. Combined with trajectory-level LLM generation, ROLLART can overlap LLM generation with environment interaction in a fine-grained manner. Furthermore, upon completing a trajectory, the `EnvManager` immediately invokes a reward computation function via a serverless API as a non-blocking task, allowing reward computation to overlap with ongoing rollouts. Overall, this trajectory-level rollout management enables a high degree of parallelism to maximize throughput.

Optimization: Redundant Env Rollouts. The trajectory-level design of `LLMProxy` and `EnvManager` enables an optimization we term *redundant environment rollouts*. This technique allows users to launch more environments than strictly required to interact with the agent LLM. Once the target number of trajectories has been collected, any ongoing rollouts can be terminated and even aborted. Because rollouts are managed at the trajectory level, slow environments do not block or delay faster ones. This effectively mitigates fail-slow and fail-stop environments. Our empirical analysis in §7.4 reveals that this technique improves rollout efficiency. Furthermore, large-scale deployments in §8 benefit from this by avoiding

the impact of environment failures.

Asynchronous Training Workflow. Figure 8 shows how ROLLART orchestrates the asynchronous training workflow. In the rollout stage, multiple `EnvManagers` act independently, continuously generating trajectories and enqueue them into the `SampleBuffer`. LLM inference and training workers run on separate GPUs, enabling disaggregated training.

The training workflow periodically runs a weight synchronization protocol. It first invokes a blocking `get_batch` call to retrieve a batch of trajectories from the `SampleBuffer`. It then issues a `suspend` command to halt trajectory collection, performs a `model_update` by fetching the latest LLM and broadcasting its weights to all inference workers. This process is accelerated by the non-blocking, cross-cluster communication optimization detailed in §5.2. Subsequently, it issues a `resume` command to continue the trajectory-level rollout with the updated model. *Unfinished trajectories from the previous iteration are also reused in the current one, through KV cache recomputation*. Last, it executes `train_step` on the retrieved data. In asynchronous training mode, the training stage overlaps with the rollout stage, and ROLLART controls the staleness of a trajectory with asynchronous bound.

Asynchronous Bound. In asynchronous training, trajectory generation can be interrupted and later resumed under a newer agent LLM. Thus, a single batch of trajectories may be generated by multiple versions of LLM. The staleness introduces high variance and compromise training stability. AReaL [12] addresses this by constraining the average sample freshness within each batch to preserve model quality. Differently, ROLLART introduces an **asynchronous bound** α to regulate freshness at the per-trajectory level and to actively manage the asynchronous workflow. It is defined per trajectory as the maximum allowable gap in version numbers between the current agent LLM and the version that initiated generation of that trajectory. If the agent LLM has advanced to version n, then any trajectory in `SampleBuffer` must have been initiated by a version no older than $(n-\alpha)$. Trajectories that violate this constraint are aborted. Our empirical study (§7.2) suggests that setting the asynchronous bound to one yields a balance between training speed and stability.

6 System Architecture

We implemented ROLLART in approximately 60k lines of Python code. Figure 9 shows the architecture of ROLLART consisting of the distributed runtime layer and the resource manager. ROLLART parses user configurations and constructs the training pipeline atop the distributed runtime. The runtime includes a rollout scheduler that manages the rollout execution workflow and the `Cluster` runtime that performs distributed computation and communication. The resource manager allocates heterogeneous resources from a shared pool to the workers.

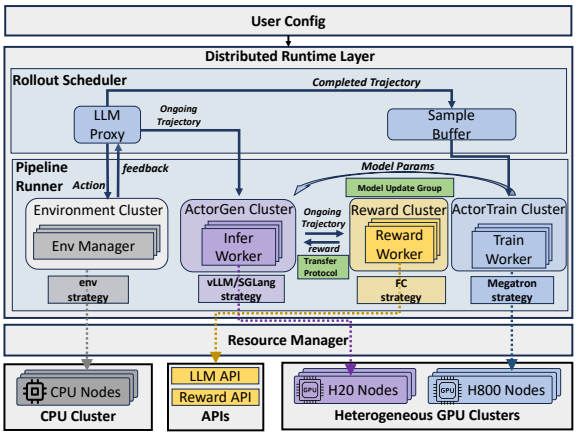


Figure 9: System Architecture of ROLLART

Rollout Scheduler. It controls the rollout stage via managing the lifecycle of each trajectory. The `LLMProxy` interacts with multiple environment managers and routes ongoing trajectories to appropriate inference workers. Completed trajectories are added to `SampleBuffer` for subsequent model training.

Pipeline Runner. The pipeline runner consists of multiple `Clusters` that assume different roles in agentic RL training (e.g., reward, environment). Each `Cluster` orchestrates a collection of workers to perform a specific type of distributed computation (e.g., training, generation, environment interaction) using a chosen strategy (e.g., vLLM [28], Megatron [52], Function Compute [3]). `Clusters` exchange trajectories via Ray's `ObjRefs` [38], which enables deferred materialization and hides data transmission overhead by passing object references instead of data copies. For model weight synchronization, each `Cluster` exposes a `model_update_group` interface that leverages high-throughput transfer engines such as NCCL [39] and Mooncake [41], utilizing NVLink, InfiniBand, and Ethernet to efficiently synchronize weights.

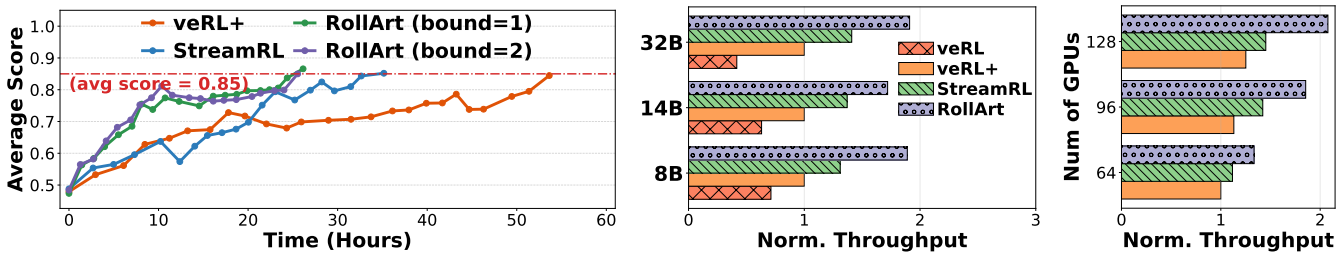
Resource Manager. The resource manager uses a persistent metadata store to maintain a real-time view of critical state, including the distributed runtime state, cluster resource availability, and service API availability. When it receives a worker deployment request, it first consults the metadata store to validate the request, then binds the requested resources or external services to the workers.

7 Performance Evaluation

In this section, we present the end-to-end evaluation (§7.2) and the analysis of three design principles (§7.3–7.5).

7.1 Evaluation Setup

Models and Tasks. We train Qwen3 [62] LLM family (8B–32B) on a diverse mixture of agentic tasks (Table 1). All models are configured with a maximum context length of 32K tokens. Due to the difficulty of the `SWE-bench` agentic



(a) Time-to-score. (b) Throughput Efficiency. (c) Resource Scaling.

Figure 10: Comparison of (a) end-to-end time-to-score on Qwen3-32B; (b) normalized throughput across LLMs for different approaches; and (c) normalized throughput of Qwen3-14B across different numbers of H800 GPUs.

task, we train it only on Qwen3-32B. we employ a 7B reward LLM to validate the reasoning process of mathematical task.

Training Configurations. We use the GRPO algorithm [21] with a training batch size of 512, a group size of 8, and a uniform sampling ratio across all tasks. For asynchronous training, we allocate 32 H800 GPUs to the training stage and use the remaining H20 and H800 GPUs for rollouts. During rollout, the tensor-parallelism degrees for Qwen3-8B/14B/32B are set to 1, 2, and 4, respectively, and we tune the training parallelism to maximize throughput. We run rollouts on vLLM 0.8.4 and training on Megatron v0.12.2, with prefix caching and CUDA graphs enabled during rollout.

Weight Update Engine. We use NCCL [39] v2.26.5 to perform intra-cluster weight update and Mooncake v0.3.7 [41] storage server for cross-cluster communication.

Hardwares. ROLLART is deployed on an H800 cluster with 96 GPUs and an H20 cluster with 32 GPUs. Within each cluster, GPU nodes are connected via 400 Gbps InfiniBand, while cross-cluster communication uses a 200 Gbps Ethernet network. We use a dedicated CPU cluster for `SWE-bench` and another for the remaining environments. The reward workers run on our internal serverless platform. Without clarification, all experiments are performed with 128 GPUs.

Baselines. Since no existing open-source system supports our full spectrum of agentic tasks, we follow the synchronous RL implementation of veRL (**veRL**). To strengthen this baseline, we additionally enable asynchronous reward computation, asynchronous environment interaction, and *Reward-as-a-Service*, and denote this as **veRL+**. We also compare against StreamRL [69] and enable the one-off training paradigm [36] (see Figure 2-Left), which parallelizes rollout and training by consuming trajectories generated in the previous step. The optimizations used in veRL+ are also applied to StreamRL. We use Megatron [52] for the training stage, which does not support heterogeneous GPU configurations. StreamRL likewise does not support heterogeneous GPUs during rollout. For simplicity, we run both baselines on 128 H800 GPUs; consequently, ROLLART incurs roughly 83% of the baselines' per-GPU-hour cost.

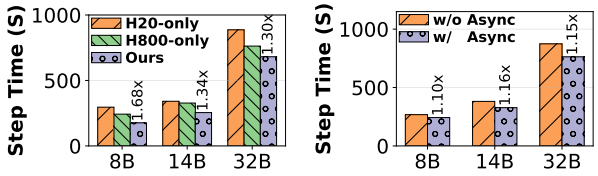
Metrics. We measure end-to-end latency as the average step time over five iterations. The throughput is the total number of prompt and response tokens in a global batch by the step time [51]. We report average validation score across all tasks.

7.2 End-to-End Evaluation

Model Convergence. We measure the score per ten iterations and report time-to-score with a target score of 0.85 in Figure 10a. StreamRL achieves a 1.52× end-to-end latency speedup over the veRL+ baseline by overlapping training and rollout. ROLLART keeps the rollout GPUs saturated by launching more rollouts and reusing partially generated trajectories across iterations. Consequently, with a bound of 1, the asynchronous approach delivers a 2.05× and 1.35 × step time reduction over the veRL+ and StreamRL. Although the asynchronous configuration with a bound of 2 exhibits a faster initial convergence rate, it results in a slightly worse time-to-score than the bound of one at later stages. Overall, different bounds still provide satisfactory convergence performance.

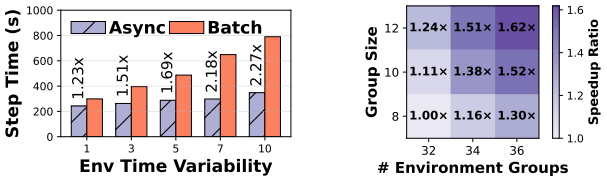
Throughput Efficiency. We present the throughput efficiency of different approaches in Figure 10b. We normalize the throughput results to veRL+ baseline. We set the bound to 1 for ROLLART. The optimization techniques including asynchronous reward and envinvornment interaction as well as serverless reward worker can improve 1.40-2.40× throughput. By overlapping rollout and training stage, we can see StreamRL achieves 1.31-1.47× throughput increase. By fine-grained ascynrhony, ROLLART can provide 2.65-4.58× throughput over the Sync baseline. Overall, ROLLART achieves substantial advantages in throughput efficiency.

Resource Scaling. We further evaluate the resource scaling performance of ROLLART. Specifically, we conduct experiments on 128 H800 GPUs and vary the number of GPUs allocated to rollout from 64 to 128 for different approaches. Figure 10c reports the normalized throughput to veRL+ baseline running on 64 H800 GPUs. As the number of allocated GPUs increases, the marginal throughput gains diminish for veRL+ and StreamRL. However, the asynchronous RL training in ROLLART continues to yield 1.33-2.08× throughput improvements, demonstrating superior resource scaling efficiency.



(a) Rollout Efficiency. (b) Cross-Cluster Comm.

Figure 11: **[Principle 1]**: (a) The efficiency of hardware affinity; (b) The benefit of async cross-cluster communication.



(a) Env Time Variability. (b) Redundant Env Rollout.

Figure 12: **[Principle 2]**: Benefits of trajectory-level Env.

7.3 Analysis of Hardware Affinity

Training Efficiency. We evaluate the training efficiency on compute-optimized and bandwidth-optimized hardware across LLMs. To isolate the impact of hardware affinity, we fix the training allocation to 32 H800 GPUs and use three resource configurations for rollout: 72 H800 GPUs as the H800-only baseline, 208 H20 GPUs as the H20-only baseline, and an affinity-aware setting with 64 H800 GPUs plus 24 H20 GPUs. In the affinity-aware setup, mathematical and game-oriented agentic tasks are prioritized routed to H20 GPUs. As shown in Figure 11a, ROLLART achieves a 1.30–1.68× step time speedup across LLM sizes compared to H20-only configuration and 1.12-1.37 speedup compared to H800-only configuration due to more H20 GPU scan reduce resource contention and benefit to decoding heavy tasks. The H20-only configuration performs worst, suggesting that many agentic tasks benefit more from compute-optimized GPUs due to frequent prefill operations. Overall, exploiting the complementary strengths of heterogeneous hardware for rollout is crucial for improving training efficiency.

Async Cross-cluster Communication. Hardware affinity introduces cross-cluster weight update, where Ethernet-connected clusters incur high communication overhead. In contrast to veRL's NCCL-based communication, which assumes that all GPU servers are connected with uniformly high-bandwidth links, our setting must explicitly handle heterogeneous interconnects. Figure 11b compares the end-to-end step time of our asynchronous cross-cluster communication with veRL's approach. The asynchronous communication technique achieves a 1.10-1.16× reduction in end-to-end step time across LLMs, indicating that cross-cluster communication overhead deserves dedicated optimization.

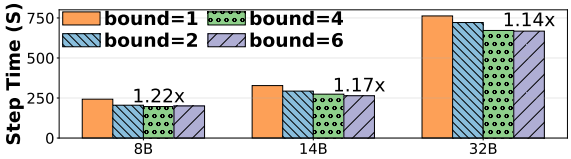


Figure 13: **[Principle 2]**: The average step time comparison of different asynchronous bounds across LLMs.

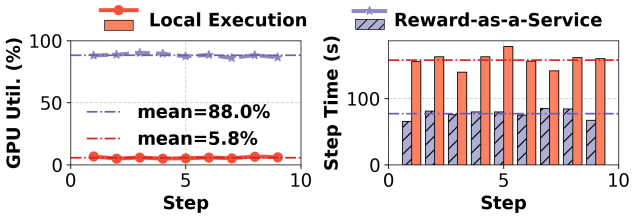


Figure 14: **[Principle 3]**: Comparison between dedicating local GPUs and using Reward-as-a-Service.

7.4 Analysis of Trajectory-level Asynchrony

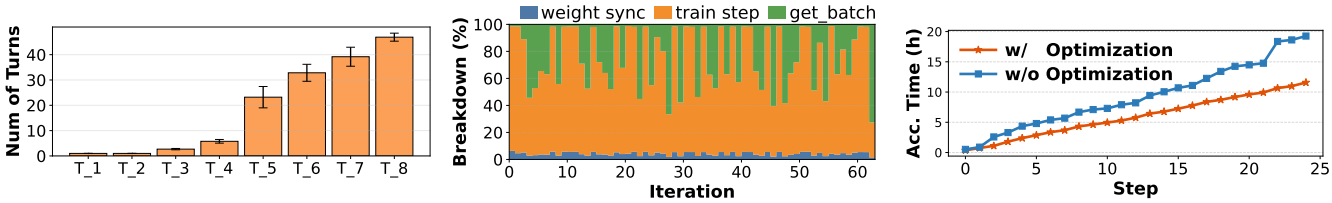
Asynchronous Environment Interaction. Trajectory-level environment management enables asynchronous environment interaction. We run Qwen3-8B/32K and inject additional environment latency sampled from gaussian distributions with mean $\mu = 10$ and standard deviation σ ranging from 1 to 10 at each turn. We compare trajectory-level and batch-level interaction in Figure 12a, reporting the average step time over ten iterations. As the latency variance increases, the performance gains of trajectory-level over batch-level interaction grows from 1.23× to 2.27×, proving asynchronous environment interaction sustains high throughput.

Redundant Environment Rollouts. GRPO exposes two key configuration parameters: the number of environment groups and the group size. By launching additional environments, redundant rollouts can tolerate environment failures and accelerate training. We run Qwen3-8B/32K on 32 H800 GPUs and vary both the number of environment groups and the group size on the `GEM-math` agentic task, reporting rollout speedup ratios in Figure 12b. The maximum speedup reaches 1.62×, and increasing either parameter yields positive speedup.

Impact of Asynchrony Bound. We vary the asynchronous bound from 1 to 6 and report the average step time across LLMs in Figure 13. Increasing the bound typically reduces the probability of aborting completed trajectories due to staleness, which in turn lowers the step time in most cases. However, we observe that efficiency plateaus within certain ranges. The optimal bound differs across LLMs and yields at most a 1.22× step time reduction compared with a bound of 1. Considering overall training performance, these throughput gains do not necessarily translate into better time-to-score. In practice, setting the bound to one delivers satisfactory performance.

7.5 Benefits of Serverless Reward Worker

We evaluate *Reward-as-a-Service* by comparing it with a dedicated local GPU setup. We run three concurrent agentic RL



(a) Average Number of Turns per Task. (b) Step Time Distribution. (c) Accumulated Step Time.

Figure 15: Production-Grade Agentic RL Workload Characterization.

jobs on mathematical tasks with Qwen3-8B/16K as the agent and Qwen2.5-7B as the reward LLM on a 16-GPU cluster, using eight GPUs for training. In the local setup, four GPUs are reserved for the reward LLM. Figure 14 shows GPU utilization and rollout time per step for a batch size of 84, where rollout time includes asynchronous reward computation. The serverless platform is shared by three jobs and perform autoscaling, increasing average GPU utilization from 6% to 88%. Without the hot-standby GPUs for reward computation, more GPUs can be utilized for rollouts. Thus, the average rollout time reduces from 158 seconds to 77 seconds, demonstrating the benefits of statefulness-aware computation.

8 RollArt in Production

Over the past six months, thousands of agentic RL jobs have used RollArt for post-training. To further demonstrate its scalability and robustness, we share our experience on a large production cluster with more than 3,000 GPUs.

Workload Characterization We trained an MoE LLM (hundreds of billions of parameters) using RollArt over in-house datasets for agentic tasks like mathematics and software engineering. The maximum prompt length and response length are 12k and 46k respectively. The average number of turns per task varies from one to 48 (Figure 15a), confirming the coexistence of joint prefill-heavy and decode-heavy tasks in agentic RL training. *The large number of turns requires a highly stable environment and fast prefill computation.*

This job adopts asynchronous training with a 1:5 ratio of training to generation GPUs. To balance gradient stability and rollout efficiency, an asynchronous bound of one is used, leading to maximum iteration time of 1.5 hours (see Figure 15b for iteration time distributions). The primary bottleneck is the blocking `get_batch` call, after the training stage finishes computing log probabilities and gradients. It waits for enough trajectories in `SampleBuffer` to reach the batch size. This consumes up to 62% of the iteration time with GPU idleness, and eliminating it could ideally reduce end-to-end training time by 22% (Figure 15b). *The training stage may stall due to insufficient trajectories from the rollout stage, making it necessary to balance the throughput of both stages.*

Characterization-driven Optimization. Informed by the workload characterization, we adjust the resource allocation ratio and optimize the prefix caching for the MoE architecture. Figure 15c shows that the characterization-drive optimization achieves 1.66× end-to-end speedup over the first 25 steps. Beyond this, we also provide dedicated optimizations for environments and system resilience.

Optimizing Environment Stability. Operating thousands of concurrent, Docker-based agentic environments on Kubernetes necessitates critical optimizations to ensure stability and performance. To combat network instability and pull overhead in `env.reset`, we employ a multi-tiered caching architecture. The first tier consists of an internal image registry that acts as a local mirror, eliminating external network calls. A second-tier, distributed load-balanced cache is situated between the compute nodes and this registry to efficiently manage high-volume requests. During an `env.reset` operation, clients prioritize fetching the required Docker image from this cache. This approach dramatically enhances the robustness of environment setup, producing above 99.99% success rate for `env.reset`.

System Resilience. We enhance system resilience at both the network and system levels. To reduce timeouts when connecting to environments, we use a persistent session-based protocol instead of a request-based one and adopt a exponential backoff retries to handle temporary network issues. Our disaggregated architecture isolates failures so that a problem in one environment, reward, or inference worker does not affect the others. Kubernetes manages environment workers, and the serverless platform manages reward workers. When an inference worker fails, it is first restarted on the same GPU. If it fails again, it is removed, and its trajectories, stored in the storage engine, are resumed on healthy workers. If a training worker fails, we restart from the latest checkpoint. This design has proven highly robust, with only a single failure observed during a week-long training run.

9 Related Work

RL Post-Training Systems. Many systems address the systems challenges of RL post-training. Early frameworks [19,24, 64] adopt static, stage-specific GPU partitions, which lowers overall utilization when the workload balance shifts across stages. veRL [50] instead uses a hybrid controller that co-locates multiple stages on the same GPUs to improve hard-

ware efficiency. Subsequent systems [6, 7, 12, 17, 20, 31, 47, 49, 69, 70] explore a range of acceleration techniques, including speculative decoding, fusion of pipeline stages to reduce orchestration overhead, asynchronous or decoupled training to hide latency, and various forms of resource disaggregation. ROLLART builds on the disaggregated and asynchronous paradigm and assigns workloads based on hardware affinity.

Resource Disaggregation. The concept of resource disaggregation, first proposed in early architectural work on memory [30], has become a cornerstone of modern system design. Recent systems like LegoOS [46] and Mira [16] decouple hardware into separate resource pools that can be managed independently to improve utilization. This pattern is prevalent in LLM serving, where systems commonly disaggregate the prefill and decoding phases [22, 40, 41, 56, 68], with some even disaggregating transformer sub-modules like attention and FFNs [55, 71]. In the context of RL post-training, several works apply a similar principle, separating the training and rollout stages across different GPU types [59, 69]. Compared to these approaches, ROLLART provides a more general and fine-grained disaggregation model, tailored specifically for the entire lifecycle of multi-task agentic RL.

10 Conclusion

In this paper, we design an efficient and scalable disaggregated RL training system. We introduce its programming model, computation model, and system architecture, all guided by three core design principles. Our microbenchmarks and macrobenchmarks demonstrate that ROLLART delivers substantial improvements in resource efficiency, scalability, and system resilience in production-level clusters.

References

[1] Introducing openai o3 and o4-mini. https://openai.com/index/introducing-o3-and-o4-mini/, 2024.

[2] Alibaba. Rtp-llm. https://github.com/alibaba/rtp-llm. Accessed: 2025-12.

[3] Alibaba Cloud. Alibaba Cloud Function Compute. https://www.alibabacloud.com/en/product/function-compute. Accessed: 2025-12.

[4] The Kubernetes Authors. Kubernetes, 2025.

[5] Axon-RL. Gem: Generalist environment for multi-task learning.

[6] Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. Respec: Towards optimizing speculative decoding in reinforcement learning systems, 2025.

[7] Rongxin Cheng, Kai Zhou, Xingda Wei, Siyuan Liu, Mingcong Han, Mingjing Ai, Yeju Zhou, Baoquan Zhong, Wencong Xiao, Rong Chen, and Haibo Chen. Fast llm post-training via decoupled and best-of-n speculation, 2025.

[8] Alibaba Cloud. Qwen model repos, 2025.

[9] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[10] DeepSeek-AI. Deepseek-v3.2-speciale. https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Speciale, 2025. Accessed: 2025-12.

[11] Farama Foundation. Gymnasium - frozenlake environment. https://gymnasium.farama.org/environments/toy_text/frozen_lake/, 2024. Accessed: 2025-09.

[12] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025.

[13] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-Latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153, Santa Clara, CA, July 2024. USENIX Association.

[14] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training, 2025.

[15] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

[16] Zhiyuan Guo, Zijian He, and Yiying Zhang. Mira: A program-behavior-guided far memory system. In *sosp*, 2023.

[17] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, Huazhong Ji, Wenjie Liu, Yu Huang, Yixiang Zhang, Chenyi Pan, Jing Wang, Xin Huang, Chunsheng Li, and Jianping Wu. Asyncflow: An asynchronous streaming rl framework for efficient llm post-training, 2025.

[18] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Yicheng Chen, Cunyin Peng, Jinjie GU, and Chenyi Zhuang. Exploring superior function calls via reinforcement learning, 2025.

[19] Eric Harper, Somshubra Majumdar, Oleksii Kuchaiev, Li Jason, Yang Zhang, Evelina Bakhturina, Vahid Noroozi, Sandeep Subramanian, Koluguri Nithin, Huang Jocelyn, Fei Jia, Jagadeesh Balam, Xuesong Yang, Micha Livne, Yi Dong, Sean Naren, and Boris Ginsburg. NeMo: a toolkit for Conversational AI and Large Language Models, 2025.

[20] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. History rhymes: Accelerating llm reinforcement learning with rhymerl, 2025.

[21] Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, et al. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025.

[22] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.

[23] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[24] Jian Hu, Xibin Wu, Weixun Wang, Dehao Zhang, Yu Cao, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.

[25] Pengcheng Jiang, Jiacheng Lin, Lang Cao, Runchu Tian, SeongKu Kang, Zifeng Wang, Jimeng Sun, and Jiawei Han. Deepretrieval: Hacking real search engines and retrievers with large language models via reinforcement learning. *CoRR*, abs/2503.00223, March 2025.

[26] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.

[27] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning, 2025.

[28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[29] Thinking Machines Lab. Tinker, 2025.

[30] Kevin Lim, Jichuan Chang, Trevor Mudge, Parthasarathy Ranganathan, Steven K Reinhardt, and Thomas F Wenisch. Disaggregated memory for expansion and sharing in blade servers. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, pages 267–278, 2009.

[31] Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Anxiang Zeng, and Jinsong Su. Specrl: Accelerating on-policy reinforcement learning via speculative rollouts, 2025.

[32] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners, 2025.

[33] Han Lu, Zichen Liu, Shaopan Xiong, Yancheng He, Wei Gao, Yanan Wu, Weixun Wang, Jiashun Liu, Yang Li, Haizhou Zhao, Ju Huang, Siran Yang, Xiaoyang Li, Yijia Luo, Zihe Liu, Ling Pan, Junchi Yan, Wei Wang, Wenbo Su, Jiamang Wang, Lin Qu, and Bo Zheng. Part ii: Roll flash – accelerating rlvr and agentic training with asynchrony, 2025.

[34] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanjing Xiong, and Hongsheng Li. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning, 2025.

[35] Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpay Ariyak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. Deepswe: Training a state-of-the-art coding agent from scratch by scaling rl, 2025. Notion Blog.

[36] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl, 2025. Notion Blog.

[37] Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. Gui-r1 : A generalist r1-style vision-language action model for gui agents, 2025.

[38] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, Carlsbad, CA, October 2018. USENIX Association.

[39] NVIDIA Corporation. NVIDIA Collective Communication Library (NCCL). https://github.com/NVIDIA/nccl. Accessed: 2025-12.

[40] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *isca*, 2024.

[41] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Kimi's kvcache-centric architecture for llm serving. *arXiv preprint arXiv:2407.00079*, 2024.

[42] Alibaba Qoder. Qoder: Agentic coding platform for real software, 2025.

[43] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[44] ByteDance Seed, Yufeng Yuan, Yu Yue, Mingxuan Wang, Xiaochen Zuo, Jiaze Chen, Lin Yan, Wenyuan Xu, Chi Zhang, Xin Liu, et al. Seed-thinking-v1. 5: Advancing superb reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.13914*, 2025.

[45] SGLang Team. Sglang: Fast serving framework for large language models. https://github.com/sgl-project/sglang, 2025. Version 0.4.

[46] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In *osdi*, 2018.

[47] Zelei Shao, Vikranth Srivatsa, Sanjana Srivastava, Qingyang Wu, Alpay Ariyak, Xiaoxia Wu, Ameen Patel, Jue Wang, Percy Liang, Tri Dao, Ce Zhang, Yiying Zhang, Ben Athiwaratkun, Chenfeng Xu, and Junxiong Wang. Beat the long tail: Distribution-aware speculative decoding for rl training, 2025.

[48] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[49] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. Laminar: A scalable asynchronous rl post-training framework, 2025.

[50] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. verl: Volcano engine reinforcement learning for llm. https://github.com/volcengine/verl, 2024.

[51] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *ACM EuroSys*, 2025.

[52] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[53] Guijin Son, Hyunwoo Ko, Hoyoung Lee, Yewon Kim, and Seunghyeok Hong. Llm-as-a-judge and reward model: What they can and cannot do, 2024.

[54] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning, 2025.

[55] StepFun. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding, 2025.

[56] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving. In *icml*, 2024.

[57] Sijun Tan, Michael Luo, Colin Cai, Tarun Venkat, Kyle Montgomery, Aaron Hao, Tianhao Wu, Arnav Balyan, Manan Roongta, Chenguang Wang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. rllm: A framework for post-training language agents, 2025. Notion Blog.

[58] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng

Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2025.

[59] Jinghui Wang, Shaojie Wang, Yinghan Cui, Xuxing Chen, Chao Wang, Xiaojiang Zhang, Minglei Zhang, Jiarong Zhang, Wenhao Zhuang, Yuchen Cao, Wankang Bao, Haimo Li, Zheng Lin, Huiming Wang, Haoyang Huang, Zongxian Feng, Zizheng Zhan, Ken Deng, Wen Xiang, Huaixi Tang, Kun Wu, Mengtong Li, Mengfei Xie, Junyi Peng, Haotian Zhang, Bin Chen, and Bing Yu. Seamlessflow: A trainer agent isolation rl framework achieving bubble-free pipelines via tag scheduling, 2025.

[60] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Zihe Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library, 2025.

[61] Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing LLM reasoning with agentic tools. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28489–28503, Vienna, Austria, July 2025. Association for Computational Linguistics.

[62] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.

[63] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023.

[64] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song, and Yuxiong He. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales, 2023.

[65] Chengyue Yu, Siyuan Lu, Chenyi Zhuang, Dong Wang, Qintong Wu, Zongyue Li, Runsheng Gan, Chunfeng Wang, Siqi Hou, Gaochi Huang, Wenlong Yan, Lifeng Hong, Aohui Xue, Yanfeng Wang, Jinjie Gu, David Tsai, and Tao Lin. Aworld: Orchestrating the training recipe for agentic ai, 2025.

[66] Minchen Yu, Ao Wang, Dong Chen, Haoxuan Yu, Xiaonan Luo, Zhuohao Li, Wei Wang, Ruichuan Chen, Dapeng Nie, Haoran Yang, and Yu Ding. Torpor: Gpu-enabled serverless computing for low-latency, resource-efficient inference. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 597–612, 2025.

[67] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. Blitzscale: Fast and live large model autoscaling with o(1) host caching, 2025.

[68] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024.

[69] Yinmin Zhong, Zili Zhang, Xiaoniu Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, Hongyu Zhou, Yimin Jiang, Yibo Zhu, and Daxin Jiang. Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation, 2025.

[70] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, and Xin Jin. Rlhfuse: Efficient rlhf training for large language models with inter- and intra-stage fusion, 2024.

[71] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, and Xin Liu. Megascale-infer: Serving mixture-of-experts at scale with disaggregated expert parallelism, 2025.

[72] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. https://github.com/THUDM/slime, 2025. GitHub repository. Corresponding author: Xin Lv.